# MDM Practical Exam

*Mobile & Wireless Communication - Quick Guide*

## Experiments covered

Exp 1 - EM Spectrum aur Wireless Bands (Case Study)

Exp 2 - Frequency Reuse (Cellular System)

Exp 7 - Multipath Effects (Signal Fluctuation)

Exp 8 - Noise aur Interference Analysis

Exp 10 - Basic Android App: Calculator

# MDM Practical Exam - Quick Guide (Exp 1, 2, 7, 8, 10)

Setup (ek baar): Python 3 + `pip install matplotlib numpy`. Phir `python3 exp1_spectrum.py` jaise chalao. Exp 10 ke liye Android Studio.

## Exp 1 - EM Spectrum aur Wireless Bands (Case Study)

**Aim: Electromagnetic spectrum ko draw karna aur dikhana ki kaun si band kis wireless communication me use hoti hai.**

### Theory (minimum)

- Frequency (f) aur wavelength (lambda) ka relation: c = f x lambda, c = 3 x 10^8 m/s. Frequency badhti hai to wavelength ghatti hai.
- Spectrum ko bands me baanta gaya hai: ELF/VLF, LF, MF, HF, VHF, UHF, SHF, EHF, phir Infrared, Visible, UV, X-ray, Gamma.
- Radio communication me mainly LF se EHF (3 kHz - 300 GHz) use hota hai. Isko ITU regulate karta hai, India me WPC/DoT licence deta hai.
- Low frequency = door tak jaati hai, walls paar karti hai, par data rate kam. High frequency = zyada data (speed) par range kam (mmWave 5G).
- Frequency log-scale pe draw karte hain kyunki range bahut badi hai (10^3 se 10^21 Hz).

| Band | Frequency | Use |
|---|---|---|
| MF | 300 kHz - 3 MHz | AM Radio |
| VHF | 30 - 300 MHz | FM Radio, TV |
| UHF | 300 MHz - 3 GHz | GSM (900/1800), 4G, TV, Bluetooth/Wi-Fi 2.4 GHz |
| SHF | 3 - 30 GHz | Wi-Fi 5 GHz, Satellite, 5G mid-band, Radar |
| EHF | 30 - 300 GHz | 5G mmWave |

### Code (exp1_spectrum.py)

```
# Exp 1: EM spectrum & wireless bands (log-scale chart)
import matplotlib.pyplot as plt

bands = [  # (naam, start Hz, end Hz, use)
   ("ELF/VLF",  3e3,   3e4,   "Submarine"),
   ("LF",       3e4,   3e5,   "Navigation"),
   ("MF",       3e5,   3e6,   "AM Radio"),
   ("HF",       3e6,   3e7,   "Shortwave"),
   ("VHF",      3e7,   3e8,   "FM, TV"),
   ("UHF",      3e8,   3e9,   "GSM, 4G, TV"),
   ("SHF",      3e9,   3e10,  "Wi-Fi, 5G, Sat"),
   ("EHF",      3e10,  3e11,  "mmWave 5G"),
   ("Infrared", 3e11,  4e14,  "Remote"),
   ("Visible",  4e14,  8e14,  "Light"),
   ("UV",       8e14,  3e16,  "Sterilise"),
   ("X-ray",    3e16,  3e19,  "Medical"),
   ("Gamma",    3e19,  1e21,  "Nuclear"),
]
fig, ax = plt.subplots(figsize=(13, 4))
ax.set_xscale("log"); ax.set_xlim(1e3, 1e21)
```

```
for i, (n, f1, f2, use) in enumerate(bands):
    ax.barh(0, f2 - f1, left=f1, color=plt.cm.rainbow(i / len(bands)), edgecolor="k")
    ax.text((f1 * f2) ** 0.5, 0, f"{n}\n{use}", ha="center", va="center", fontsize=7, rotation=90)
for f, n, h in [(9e8, "GSM 900", 0.85), (2.4e9, "Wi-Fi/BT 2.4G", 0.65), (5e9, "Wi-Fi 5G", 0.85)]:
    ax.annotate(n, (f, 0.5), (f, h), ha="center", fontsize=7, arrowprops=dict(arrowstyle="->"))
ax.set_yticks([]); ax.set_ylim(-0.6, 1.1)
ax.set_xlabel("Frequency (Hz) - log scale"); ax.set_title("Electromagnetic Spectrum & Wireless Bands")
plt.tight_layout(); plt.savefig("exp1_spectrum.png", dpi=150); plt.show()
```

Kaise chalana hai: python exp1_spectrum.py -> chart window khulega aur exp1_spectrum.png save hoga.

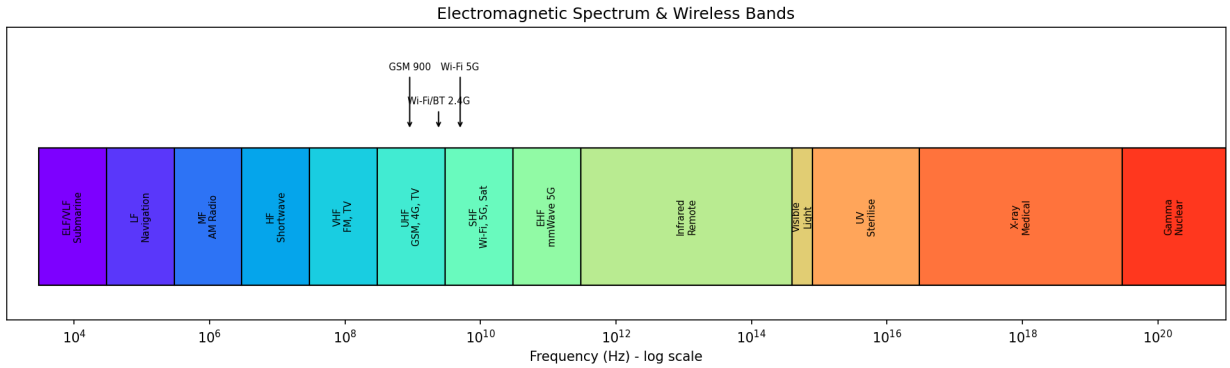



*Fig 1: EM spectrum (log scale) with wireless bands marked*

### Viva ke liye (Q & A)

**Q. Spectrum log-scale pe kyun?**

A. Kyunki frequencies 10^3 se 10^21 tak phaili hain; linear me chhoti bands dikhengi hi nahi.

**Q. Wi-Fi aur Bluetooth kis band me?**

A. 2.4 GHz (UHF/ISM) aur Wi-Fi 5 GHz (SHF). ISM band license-free hai.

**Q. 5G ko mmWave kyun bolte hain?**

A. 30-300 GHz ki wavelength 1-10 mm hoti hai. Speed zyada, range kam.

**Q. Low frequency door tak kyun jaati hai?**

A. Path loss aur obstacle absorption kam hota hai, wavelength badi hone se diffraction zyada hota hai.

## Exp 2 - Frequency Reuse (Cellular System)

**Aim: Cellular system me frequency reuse ka concept simulation se dikhana.**

### Theory (minimum)

- Spectrum limited hai, users zyada. Isliye ek hi frequency ko door-door ke cells me dobara use karte hain = frequency reuse.

- Cell = hexagon (base station ka coverage). N cells ka group = cluster; cluster ke andar sab frequencies alag hoti hain.
- Cluster size: N = i^2 + i*j + j^2  (i, j integers). Common values 3, 4, 7, 12.
- Co-channel reuse distance: D = R x sqrt(3N)  (R = cell radius). D/R = Q = reuse ratio.
- Bada N => D/R bada => interference (co-channel) kam, par har cell ko channels kam milte (1/N). Chhota N => capacity zyada par interference zyada.
- Ek hi frequency set use karne wale cells = co-channel cells.

### Code (exp2_reuse.py)

```
# Exp 2: Frequency reuse - drag sliders to change i, j  ->  N = i^2 + i*j + j^2,  Q = D/R = sqrt(3N)
import numpy as np, matplotlib.pyplot as plt
from matplotlib.patches import RegularPolygon
from matplotlib.widgets import Slider

fig, ax = plt.subplots(figsize=(8, 8.5))
plt.subplots_adjust(bottom=0.17)
s_i = Slider(plt.axes([0.2, 0.08, 0.6, 0.03]), "i", 0, 4, valinit=2, valstep=1)
s_j = Slider(plt.axes([0.2, 0.03, 0.6, 0.03]), "j", 0, 4, valinit=1, valstep=1)

def draw(_=None):
    i, j = int(s_i.val), int(s_j.val)
    if i == j == 0: return
    N = i*i + i*j + j*j; Q = np.sqrt(3*N); R = 1.0
    ax.clear(); cells, keys = [], {}
    for q in range(-12, 13):
        for r in range(-12, 13):
            x, y = np.sqrt(3)*R*(q + r/2), 1.5*R*r
            if np.hypot(x, y) > 10: continue
            k = ((q*(i+j) + r*j) % N, (r*i - q*j) % N)   # same k = same frequency set
            c = keys.setdefault(k, len(keys))
            cells.append((x, y, c))
            ax.add_patch(RegularPolygon((x, y), 6, radius=R, fc=plt.cm.gist_rainbow(c/N), ec="k", alpha=.8))
            ax.text(x, y, f"F{c+1}", ha="center", va="center", fontsize=7)
    x0, y0, c0 = next(c for c in cells if c[0] == 0 and c[1] == 0)
    near = min((c for c in cells if c[2] == c0 and c[:2] != (x0, y0)), key=lambda c: np.hypot(c[0]-x0, c[1]-y0))
    ax.plot([x0, near[0]], [y0, near[1]], "k-", lw=3)
    ax.text((x0+near[0])/2, (y0+near[1])/2, f"D={np.hypot(near[0]-x0, near[1]-y0):.2f}R", ha="center",
            fontweight="bold", bbox=dict(fc="w", ec="k"))
    ax.set_xlim(-10.5, 10.5); ax.set_ylim(-10.5, 10.5); ax.set_aspect("equal"); ax.axis("off")
    ax.set_title(f"i={i}, j={j}   N = {N}   Q = D/R = sqrt(3N) = {Q:.2f}   (same F = same frequencies)")
    fig.canvas.draw_idle()
    print(f"i={i} j={j} N={N}  Q=D/R={Q:.2f}  each cell gets 1/{N} of channels")

s_i.on_changed(draw); s_j.on_changed(draw); draw()
fig.savefig("exp2_reuse.png", dpi=120)   # picture of the starting view (i=2, j=1)
plt.show()
```

Kaise chalana hai: python3 exp2_reuse.py. Neeche 2 sliders (i aur j) hain - mouse se drag karo, N aur Q = D/R title me live badalta hai (jaise i=1,j=1 -> N=3, Q=3.00; i=2,j=1 -> N=7, Q=4.58; i=3,j=1 -> N=13, Q=6.24). Same label (jaise F1) wale cells same frequencies use karte hain. Kaali line = reuse distance D.

Q badhane / ghatane ka tareeka: Q = D/R = sqrt(3N) aur N = i^2 + i*j + j^2, isliye Q ko seedha nahi, i aur j slider se badlo. Q badhana hai to i ya j badhao (i=1,j=1: Q=3.00 -> i=2,j=1: Q=4.58 -> i=3,j=1: Q=6.24). Q ghatana hai to i ya j ghatao. Valid N sirf 3, 4, 7, 9, 12, 13... hote hain, isliye Q in values ke beech me nahi aata. Bada Q = interference kam par capacity kam; chhota Q = capacity zyada par interference zyada.

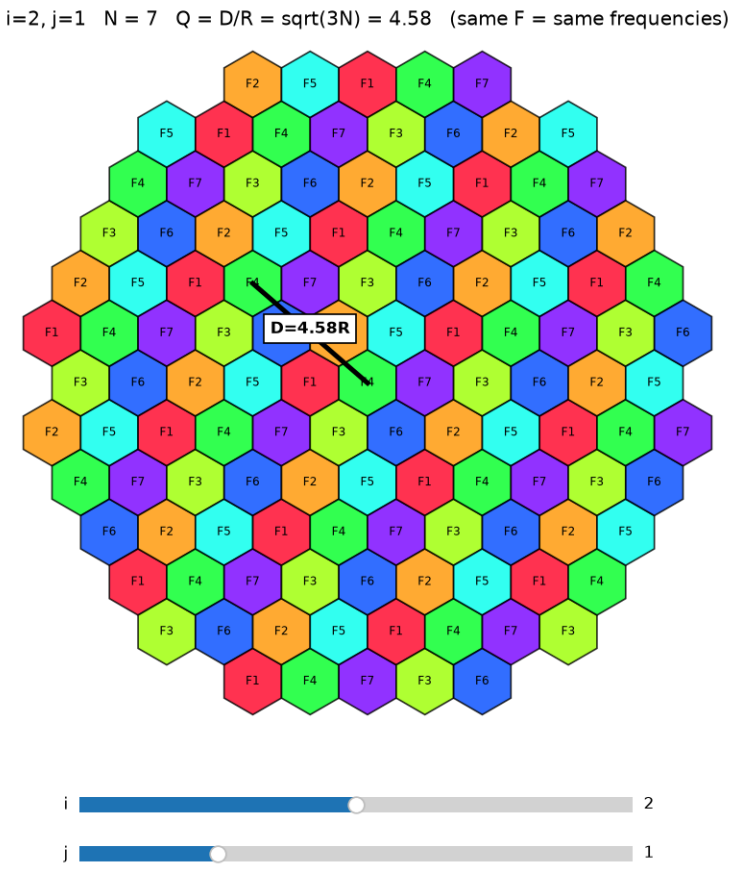



*Fig 2: Frequency reuse with N = 7. Line = D = 4.58R (same F1 cells); starting view of the sliders*

Output (terminal): N = 7 | D/R = sqrt(3N) = 4.58 | each cell gets 1/7 of the total channels

Observation table (change i, j with the sliders and read D from the graph). Result: as N increases, D/R increases and interference decreases, but each cell gets only 1/N of the channels.

| **i** | **j** | **N** | **Q = D/R** | **D (from graph)** | **Channels / cell** | **Observation** |
|---|---|---|---|---|---|---|
| 1 | 1 | 3 | 3.00 | 3.00 R | 1/3 | D small, more interference |
| 2 | 0 | 4 | 3.46 | 3.46 R | 1/4 | D slightly larger |
| 2 | 1 | 7 | 4.58 | 4.58 R | 1/7 | Standard cluster, balanced |
| 2 | 2 | 12 | 6.00 | 6.00 R | 1/12 | D large, less |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | interference |
| 3 | 1 | 13 | 6.24 | 6.24 R | 1/13 | D larger still, fewer channels per cell |

### Viva ke liye (Q & A)

**Q. N = 7 me D/R kitna?**

A. sqrt(3 x 7) = sqrt(21) = 4.58.

**Q. N badhane se kya hota hai?**

A. Interference kam, par capacity per cell kam (kyunki channels N me baant gaye).

**Q. Co-channel interference kya hai?**

A. Same frequency use karne wale doosre cell ka signal jo hamare signal me dakhal deta hai.

**Q. Q ko kaise badhaya?**

A. Q = sqrt(3N) aur N sirf i, j se milta hai, isliye i/j slider se N (aur Q) badalta hai. Q seedha continuous nahi hota, sirf valid N values (3, 4, 7, 9, 12, 13...) par.

## Exp 7 - Multipath Effects (Signal Fluctuation)

**Aim: Multipath fading ki wajah se signal ka utar-chadhav (fluctuation) simulation se dekhna.**

### Theory (minimum)

- Transmitter ka signal seedha (LOS) + deewar, building se reflect ho kar alag-alag raston (paths) se receiver tak aata hai.
- Har path ki length alag => delay alag => phase alag. Receiver sab ko jod (add) deta hai.
- Phase same => signals add (constructive) => signal strong. Phase ulta => cancel (destructive) => deep fade.
- Receiver sirf lambda/2 (900 MHz pe ~17 cm) move kare to bhi signal bahut badal jaata hai - isko small-scale fading kehte hain.
- Bahut saare paths aur no LOS => amplitude Rayleigh distribution; LOS ho to Rician.
- Effects: signal drops, ISI (delay spread), call drop. Solution: diversity antennas, equalizer, OFDM, RAKE receiver.

### Code (exp7_multipath.py)

```
# Exp 7: Multipath fading - sum of many reflected paths (Rayleigh) as receiver moves
import numpy as np, matplotlib.pyplot as plt
np.random.seed(1)
f, c = 900e6, 3e8; lam = c / f              # 900 MHz carrier
d = np.linspace(0, 5, 1000)                 # receiver moves 5 m
P = 20                                      # number of paths
amp = np.random.rand(P); ph = 2*np.pi*np.random.rand(P); ang = 2*np.pi*np.random.rand(P)
```

```
# each path has a different phase shift depending on arrival angle -> add them up
h = sum(amp[k]*np.exp(1j*(ph[k] + 2*np.pi*d/lam*np.cos(ang[k]))) for k in range(P))
rssi = 20*np.log10(np.abs(h)/np.abs(h).mean())  # dB relative to mean

plt.figure(figsize=(9, 4))
plt.plot(d, rssi); plt.axhline(-10, color="r", ls="--", label="deep fade (-10 dB)")
plt.xlabel("Distance moved (m)"); plt.ylabel("Signal level (dB)"); plt.grid(alpha=.3); plt.legend()
plt.title("Multipath Fading: 20 reflected paths, 900 MHz")
plt.tight_layout(); plt.savefig("exp7_multipath.png", dpi=150); plt.show()
print("Max fade depth: %.1f dB, fades below -10 dB: %.1f%% of the time" % (rssi.min(), 100*(rssi < -10).mean()))
```

Simple explain: 20 random paths banaye (random amplitude, phase, angle). Receiver 5 m chalta hai. Har point par saare paths ko complex number ki tarah add kiya: h = sum(a*e^(j*phase)). Phir |h| ko dB me plot kiya.

Kaise chalana hai: python exp7_multipath.py.

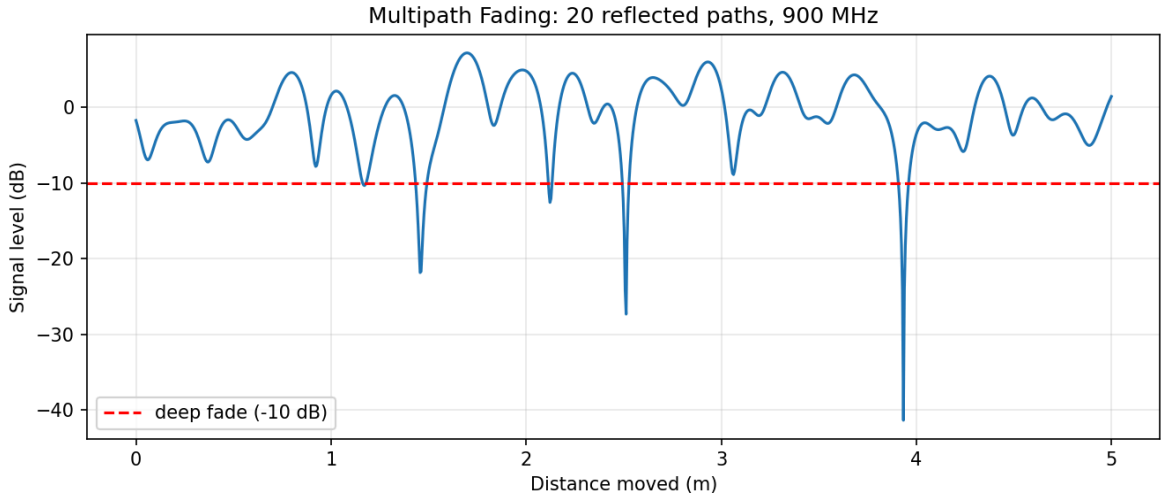



*Fig 3: Multipath fading - signal level (dB) vs distance moved. Dips below red line = deep fades (SIMULATION)*

Output (terminal, seed = 1): Max fade depth: -41.4 dB, fades below -10 dB: 3.5% of the time

Observation table (from the graph). Result: even a small movement of the receiver changed the signal by ~48 dB - this is the effect of multipath.

| **Parameter** | **Value** | **Where on the graph** |
|---|---|---|
| Max signal | +7.2 dB at 1.70 m | highest peak |
| Min signal (deepest fade) | -41.4 dB at 3.93 m | deepest dip |
| Variation (max - min) | 48.5 dB | within only 5 m of movement |
| Time below -10 dB | 3.5 % | below the red dashed line |
| Deep fades (< -10 dB) | 5 in 5 m | dips that cross the red line |

| | | |
|---|---|---|
| Wavelength (900 MHz) | 0.33 m | fade spacing is of the order of the wavelength |

### Viva ke liye (Q & A)

**Q. Multipath kya hai?**

A. Signal ka kai alag raston se receiver tak pahunchna (reflection, diffraction, scattering).

**Q. Deep fade kab hota hai?**

A. Jab saare paths ka phase aisa ho ki wo ek doosre ko cancel kar dein.

**Q. Rayleigh aur Rician me fark?**

A. Rayleigh = no LOS path. Rician = ek strong LOS path + scattered paths.

**Q. Isko kam kaise karte hain?**

A. Antenna diversity (MIMO), equalizer, OFDM, RAKE receiver, error-correcting codes.

## Exp 8 - Noise aur Interference Analysis

**Aim: Noise aur interference ka wireless link ki quality (BER) par asar simulation se dekhna.**

### Theory (minimum)

- Noise: random unwanted signal (thermal noise, AWGN). Thermal noise power = kTB (B = bandwidth).
- Interference: doosre transmitters ka signal (same ya paas ki frequency) - co-channel / adjacent channel.
- SNR = Signal power / Noise power. SIR = Signal / Interference. Dono dB me napte hain.
- BER (Bit Error Rate) = galat bits / total bits. SNR badhne par BER girti hai.
- Interference ho to BER ek jagah 'flat' ho jaati hai - SNR badhaane se bhi zyada fayda nahi (error floor).
- Is simulation me BPSK use kiya: bit 0 -> -1, bit 1 -> +1; receiver sign dekh kar decide karta hai.

### Code (exp8_noise.py)

```
# Exp 8: Noise & interference - BER vs SNR for BPSK, with and without interferer
import numpy as np, matplotlib.pyplot as plt
np.random.seed(0)
N = 200000
bits = np.random.randint(0, 2, N); s = 2*bits - 1          # BPSK: 0 -> -1, 1 -> +1

def ber(snr_db, sir_db=None):
    noise = np.random.randn(N) / np.sqrt(2 * 10**(snr_db/10))       # AWGN
    r = s + noise
    if sir_db is not None:                                  # interfering signal
        r += 10**(-sir_db/20) * np.sign(np.random.randn(N))
    return np.mean((r > 0) != bits)

snr = np.arange(0, 11)
plt.figure(figsize=(7, 4.5))
plt.semilogy(snr, [ber(x) + 1e-6 for x in snr], "o-", label="Only noise (AWGN)")
```

```
plt.semilogy(snr, [ber(x, 6) + 1e-6 for x in snr], "s-", label="Noise + interference (SIR 6 dB)")
plt.semilogy(snr, [ber(x, 3) + 1e-6 for x in snr], "^-", label="Noise + interference (SIR 3 dB)")
plt.xlabel("SNR (dB)"); plt.ylabel("Bit Error Rate"); plt.grid(alpha=.3, which="both"); plt.legend()
plt.title("Effect of Noise & Interference on BER (BPSK)")
plt.tight_layout(); plt.savefig("exp8_noise.png", dpi=150); plt.show()
```

Simple explain: 2 lakh random bits -> BPSK -> noise add -> (optional) interferer add -> decision -> galat bits ginte hain. Ye 3 case me kiya: sirf noise, noise + interference (SIR 6 dB), noise + interference (SIR 3 dB).

Kaise chalana hai: python exp8_noise.py.

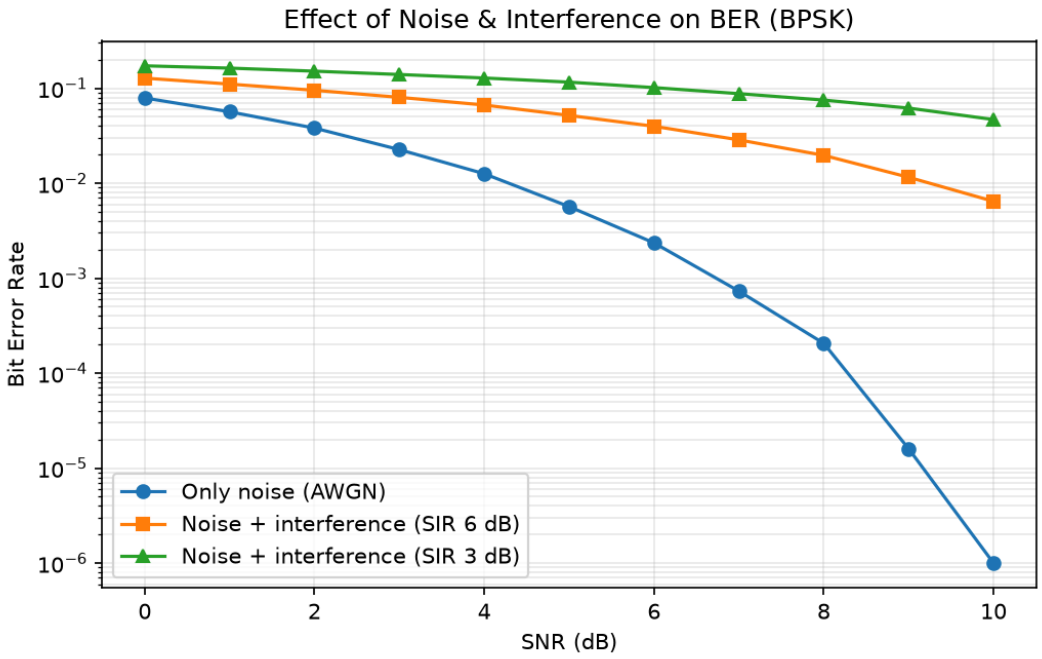



*Fig 4: BER vs SNR for BPSK (SIMULATION, 200000 bits per point)*

Result: with noise only, the BER at SNR 10 dB is ~1e-6 or even lower (the graph has a 1e-6 floor so the log plot works). With interference, the BER at 10 dB is ~6e-3 (SIR 6 dB) and ~5e-2 (SIR 3 dB), and it falls very slowly as SNR increases. So if the interference is strong, increasing SNR alone is not enough.

Observation table (read from the graph, just remember these 3 points):

| **Case** | **Curve on the graph** | **BER at SNR = 10 dB (approx)** |
|---|---|---|
| Noise only | Falls steeply | almost 0 (very low) |
| Noise + interference (SIR 6 dB) | Falls slowly | ~1 bit in 150 is wrong |
| Noise + interference (SIR 3 dB) | Stays high | ~1 bit in 20 is wrong |

### Viva ke liye (Q & A)

**Q. SNR aur SIR me fark?**

A. SNR = signal/noise; SIR = signal/interference.

**Q. BER kya hai?**

A. Galat receive hue bits ka ratio.

**Q. Interference ke saath curve lagbhag flat kyun?**

A. Interference noise ki tarah kam nahi hoti jab SNR badhta hai, isliye error floor bana rehta hai.

**Q. Kaise reduce karein?**

A. Power control, frequency reuse planning, filters, sectorization, error-correcting codes.

## Exp 10 - Basic Android App: Calculator

**Aim: Android Studio me ek basic app banana jo basic arithmetic operations (+, -, x, /, %) kare.**

### Theory (minimum)

- Android app = Activity (screen ka logic, Java file) + Layout (screen ka design, XML file) + Manifest (app ki declaration).
- Is app me XML layout nahi hai: poori screen Java code se banti hai - LinearLayout me TextView (display) aur GridLayout me Buttons (ek loop se). Har button ka click ek hi function press() ko call karta hai.
- Java me button ka text padhte hain: digit ho to number me jodte hain, operator ho to pehla number save karte hain, '=' pe calculate karte hain.
- Note: Exp 10 ka topic 'wireless communication scenario' hai. Agar examiner Wi-Fi/Bluetooth app maangta hai to meri purani Exp 10 report (Bluetooth chat) use karo; ye calculator wo hai jo tumne exam ke liye maanga.

### Android Studio me steps

- New Project -> Empty Views Activity -> Name: Calculator, Language: Java -> Finish.
- MainActivity.java ka poora content (sirf pehli package line chhod kar) neeche wale code se replace karo. Package name wahi rakhna jo tumhare project me hai.
- activity_main.xml aur themes.xml ko chhedna nahi hai - sirf ek file likhni hai.
- Run (green triangle) -> emulator ya phone.

### MainActivity.java (sirf yehi ek file likhni hai)

```
package com.vcet.calculator;

import android.graphics.Color;
import android.os.Bundle;
import android.view.Gravity;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    TextView tv;
    String cur = "";       // number being typed
```

```
    double a = 0;           // first number
    String op = "";         // + - × ÷ %
    String[] keys = {"C", "⌫", "%", "÷", "7", "8", "9", "×", "4", "5", "6", "−",
                     "1", "2", "3", "+", "0", ".", "="};

    @Override
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        if (getSupportActionBar() != null) getSupportActionBar().hide();

        LinearLayout root = new LinearLayout(this);          // whole screen (no XML needed)
        root.setOrientation(LinearLayout.VERTICAL);
        root.setBackgroundColor(0xFF121212);
        root.setFitsSystemWindows(true);
        root.setPadding(24, 24, 24, 24);

        tv = new TextView(this);                             // display
        tv.setText("0"); tv.setTextColor(Color.WHITE); tv.setTextSize(56);
        tv.setGravity(Gravity.BOTTOM | Gravity.END);
        root.addView(tv, new LinearLayout.LayoutParams(-1, 0, 1));

        GridLayout grid = new GridLayout(this);              // 4 x 5 buttons
        grid.setColumnCount(4); grid.setRowCount(5);
        root.addView(grid, new LinearLayout.LayoutParams(-1, 0, 2));

        for (String k : keys) {
            Button btn = new Button(this);
            btn.setText(k); btn.setTextSize(26);
            boolean top = k.matches("C|⌫|%"), orange = "÷×−+=".contains(k);
            btn.setTextColor(top ? Color.BLACK : Color.WHITE);
            btn.setBackgroundTintList(android.content.res.ColorStateList.valueOf(
                    orange ? 0xFFFF9F0A : top ? 0xFFA5A5A5 : 0xFF333333));
            int span = k.equals("0") ? 2 : 1;
            GridLayout.LayoutParams p = new GridLayout.LayoutParams(
                    GridLayout.spec(GridLayout.UNDEFINED, 1f),
                    GridLayout.spec(GridLayout.UNDEFINED, span, (float) span));
            p.width = 0; p.height = 0; p.setMargins(6, 6, 6, 6);
            grid.addView(btn, p);
            btn.setOnClickListener(v -> press(k));
        }
        setContentView(root);
    }

    void press(String t) {
        if (t.matches("[0-9]")) cur += t;                    // digit
        else if (t.equals(".") && !cur.contains(".")) cur += cur.isEmpty() ? "0." : ".";
        else if (t.equals("C")) { cur = ""; op = ""; a = 0; }
        else if (t.equals("⌫")) { if (!cur.isEmpty()) cur = cur.substring(0, cur.length() - 1); }
        else if (t.equals("=")) {
            if (!op.isEmpty() && !cur.isEmpty()) { cur = fmt(calc(a, Double.parseDouble(cur), op)); op = ""; }
        } else if ("+−×÷%".contains(t) && !cur.isEmpty()) {     // operator
            if (!op.isEmpty()) cur = fmt(calc(a, Double.parseDouble(cur), op));
            a = Double.parseDouble(cur); op = t; cur = "";
        }
        tv.setText(cur.isEmpty() ? (op.isEmpty() ? "0" : fmt(a) + " " + op) : (op.isEmpty() ? cur : fmt(a) + " " + op + " " + cur));
    }

    double calc(double x, double y, String o) {
        switch (o) {
            case "+": return x + y;
            case "−": return x - y;
            case "×": return x * y;
            case "÷": return y == 0 ? Double.NaN : x / y;
            default:  return x % y;
        }
    }

    String fmt(double d) {
        if (Double.isNaN(d)) return "Error";
        return d == (long) d ? String.valueOf((long) d) : String.valueOf(d);
    }
}
```

Code ka seedha matlab:

- cur = jo number type ho raha hai; a = pehla number; op = chuna hua operator.
- Digit dabaya => cur me jod do. Operator dabaya => a = cur, op = symbol, cur khaali.
- '=' dabaya => calc(a, cur, op) se answer nikalo. Division by zero => 'Error'.
- C = sab clear, backspace = last character hatao.

Test (emulator par kiya): 12 x 3 = 36 (neeche screenshot), 7 / 2 = 3.5, 5 - 9 = -4, 8 % 3 = 2, 9 / 0 = Error. Addition (+) ka alag test nahi chalaya, par wo bhi isi calc() function se jaata hai.

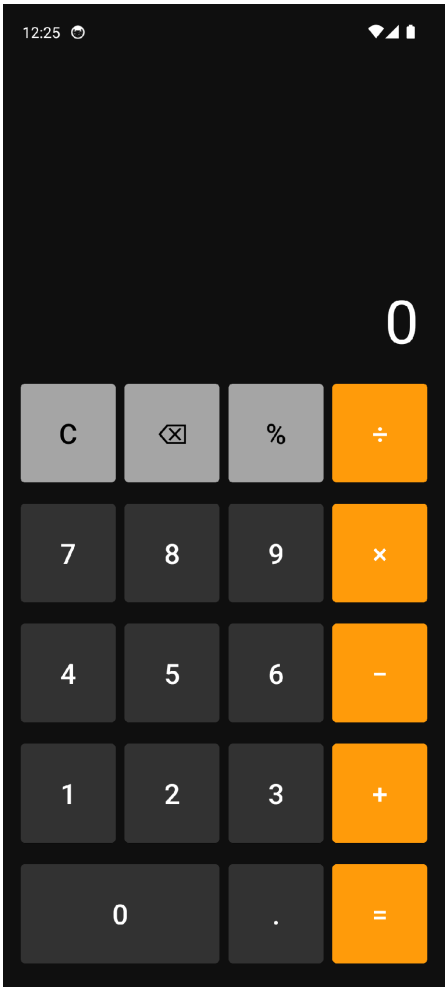



*Fig 5: Calculator app on Android 14 emulator (start screen)*

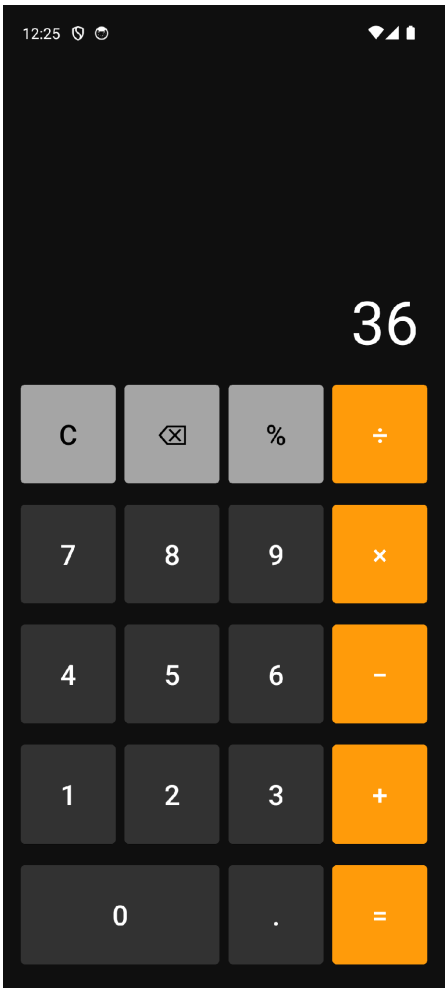



*Fig 6: 12 x 3 = 36 on the emulator (real emulator screenshot)*

## Viva ke liye (Q & A)

**Q. Activity kya hai?**

A. App ki ek screen jisme UI aur uska logic hota hai.

**Q. Button dabane par kya hota hai?**

A. Loop me har button par setOnClickListener lagaya hai; click par press(text) function chalta hai.

**Q. Manifest me kya hota hai?**

A. App ka naam, theme, permissions, aur launcher activity.

**Q. Isko Wi-Fi/Bluetooth se kaise jodein?**

A. Result ko BluetoothSocket ke OutputStream me bhej sakte hain; poora example meri Exp 10 Bluetooth chat app me hai.